Real-Time Video Generation With MiniMax H3

8× RTX PRO 5000 · vLLM-Omni

Shunyang Li | Public technical brief | October 2026



Agenda

01 Toward Real-Time H3 Generation
02 vLLM-Omni and ComfyUI Applications

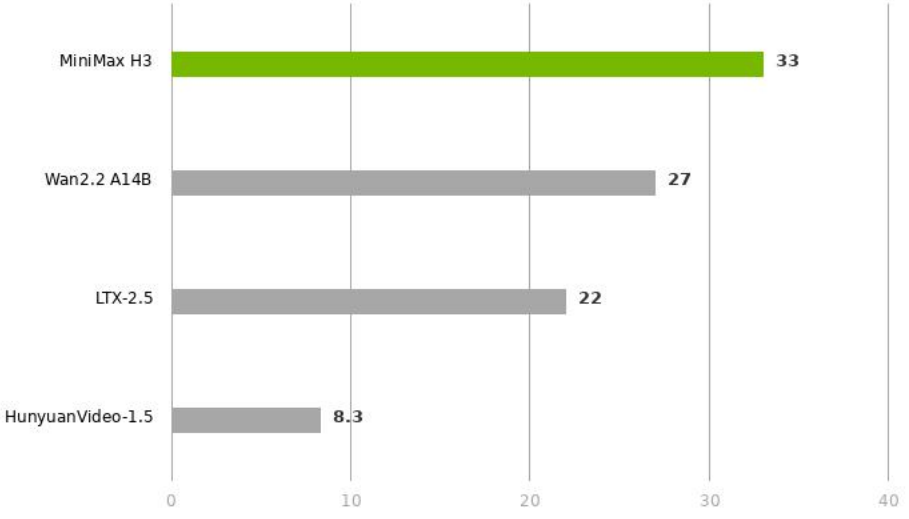


01 · Real-Time H3 Generation



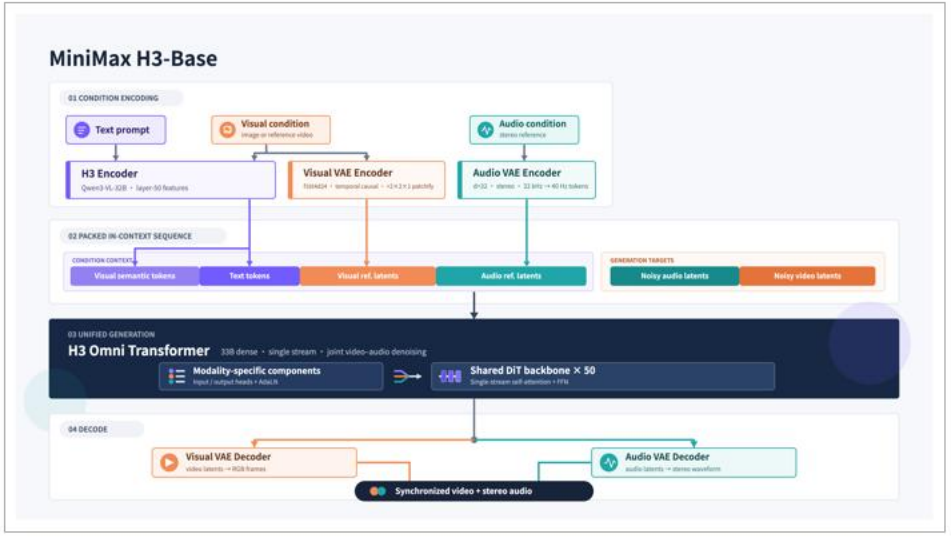
Real-Time Ref2VA Raises the Systems Bar

A 124-frame clip at 24 FPS is real time only when end-to-end latency stays below 5.167 s (RTF < 1)



Generative Backbone · Full Parameters (B)

RTF = latency ÷ media duration; the 124-frame deadline is 5.167 s. H3 has the largest dense backbone in this peer set; Wan2.2 has 27B total and 14B active parameters



Model Capability · Native 768P · 33B Dense

32B encoder → 33B dense Omni Transformer → visual/audio VAE. Fifty blocks × 50 steps keep compute and memory pressure high; about 13B AdaLN parameters can be cached

Sources: MiniMax H3, Wan2.2, LTX-2.5 and HunyuanVideo-1.5 official releases · author 960×544 Ref2VA workload

SM120 Brings FP4 Compute; PCIe Limits Scale

All discrete SM120 cards pair fifth-generation Tensor Cores with GDDR7 and PCIe Gen5; none provides GPU-to-GPU NVLink

SKU	Compute	Memory	GPU I/O / Fabric
RTX PRO 5000 ×8 Blackwell	SM120 · 110 SMs 5th-gen Tensor · FP4	72 GB GDDR7 ECC 48 GB variant	PCIe Gen5 ×16 Planned: NIC → RDMA*
RTX PRO 6000 WS / Server	SM120 · 188 SMs 5th-gen Tensor · FP4	96 GB GDDR7 ECC	PCIe Gen5 ×16 No GPU NVLink
GeForce RTX 5090	SM120 · 170 SMs 5th-gen Tensor · FP4	32 GB GDDR7	PCIe Gen5 ×16 No NVLink
DGX Spark · GB10	SM121 · 48 SMs 5th-gen Tensor · FP4	128 GB unified LPDDR5X	NVLink-C2C in SoC 200 Gb/s ConnectX-7

* Planned system path; GPUDirect RDMA requires a compatible NIC and PCIe topology · Sources: NVIDIA product specs, CUDA GPU table, GPUDirect RDMA guide

Latency Breakdown

NCCL occupies 49.2% of the average GPU timeline

Stage	Average GPU Time*	Share
Mean GPU timeline*	250.0 s	100.0%
NCCL communication	123.0 s	49.2%
↳ DiT collectives	116.1 s	46.4%
↳ Encoder, VAE, and other collectives	7.0 s	2.8%
Tensor compute	84.5 s	33.8%
↳ Attention	45.3 s	18.1%
↳ GEMM / MLP	39.2 s	15.7%
AdaLN weight staging + device layout	38.5 s	15.4%
Encoder, VAE, and other GPU work	4.0 s	1.6%

- Of the 38.5 s average, 36.9 s is AdaLN projection weight staging from CPU; layout is 1.5 s
- This run uses the default runtime cache, not a complete precomputed schedule sidecar

Base Ref2VA · 960×544 · 124f @ 24 FPS · 50 steps · BF16 · 8× RTX PRO 5000 · default AdaLN offload · local NCCL / RDMA off · *sum across 8 GPU timelines ÷ 8

How Denoising Works

H3 follows a learned velocity field from noise toward clean video and audio latents

State and direction

- x(t) is the current latent; the DiT predicts a data-ward velocity

Solver update

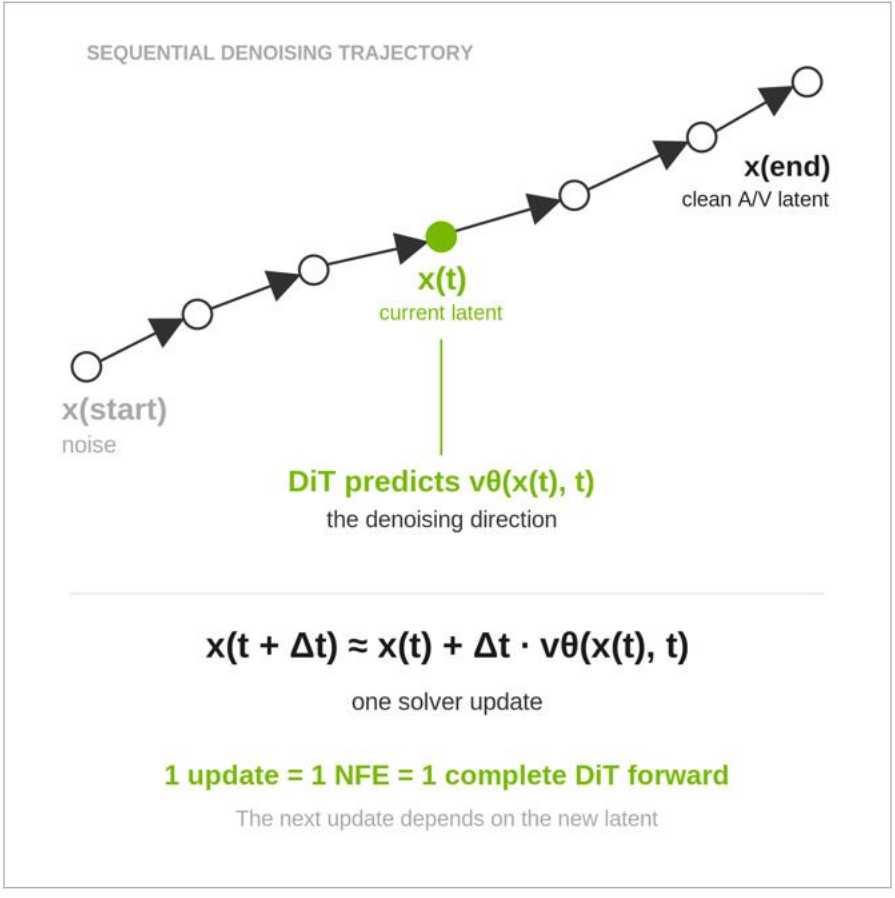
- Euler advances x(t) by a small interval along that velocity
- The next DiT call needs the new latent, so conventional steps run in sequence

Network function evaluation (NFE)

- One NFE is one complete DiT forward

Base H3

- 50 NFE × 50 main blocks = 2,500 main-block passes



Classical Flow-Matching Sampling

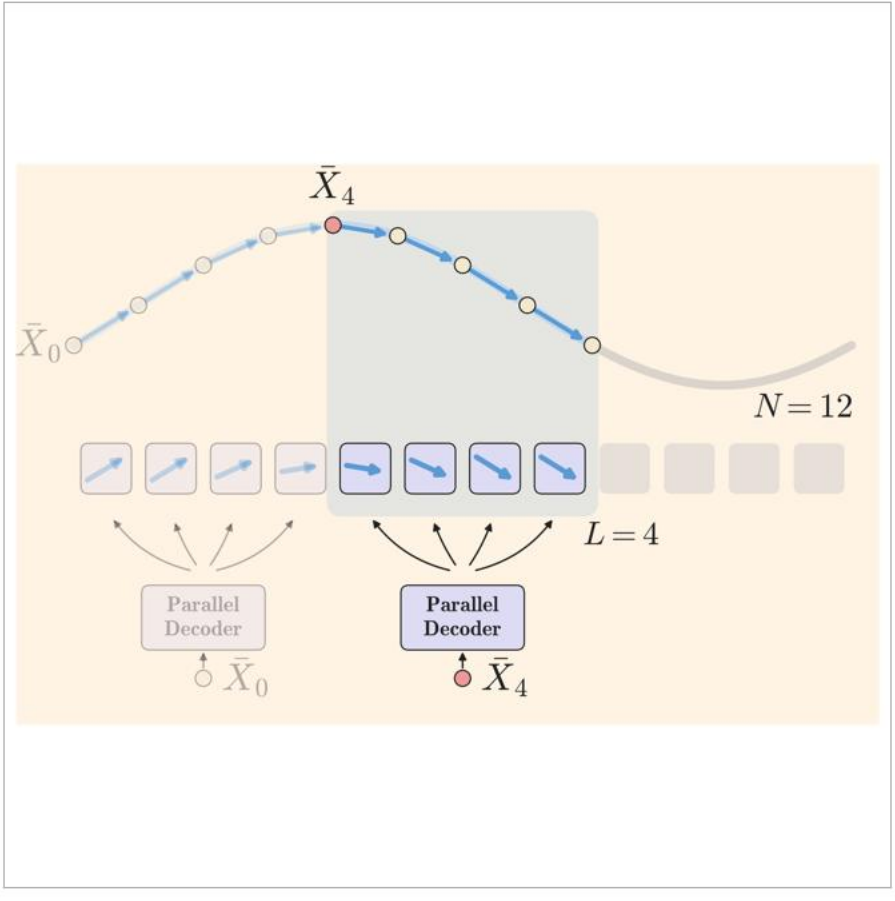
A chain of dependent solver updates turns noise into clean latents

Sources: MiniMax H3 Base configuration · NVIDIA FastGen-PDD sampling background (arXiv:2607.26004) · author abstraction

Few-Step Distillation

PDD predicts four denoising intervals per DiT call: 50 calls → 8

- Base H3 calls the full DiT 50 times. Each call predicts one short interval, updates the latent, and provides the starting point for the next call
- PDD uses Base H3 as the teacher and trains a student on groups of four adjacent denoising intervals
- One student call predicts four consecutive directions from the same noisy latent
- The solver combines those directions, advances the latent, and invokes the student again
- The released H3 schedule covers 32 fine intervals in eight student calls
- **50 → 8 DiT calls; 2,500 → 400 repeated block passes (−84%)**



N = fine intervals · L = intervals predicted in parallel

Paper example: N = 12, L = 4 · H3 PDD8: N = 32, L = 4

Sources: NVIDIA FastGen-PDD, Fig. 2 (arXiv:2607.26004) · Alibaba-PAI MiniMax-H3 Ref2VA Acc-8Step @ 335001f

Sparse Attention

Coarse attention finds relevant cubes; fine attention resolves token-level detail

Predict important regions cheaply

- Pool spatiotemporal cubes and run global attention on the shorter sequence

Select content-dependent blocks

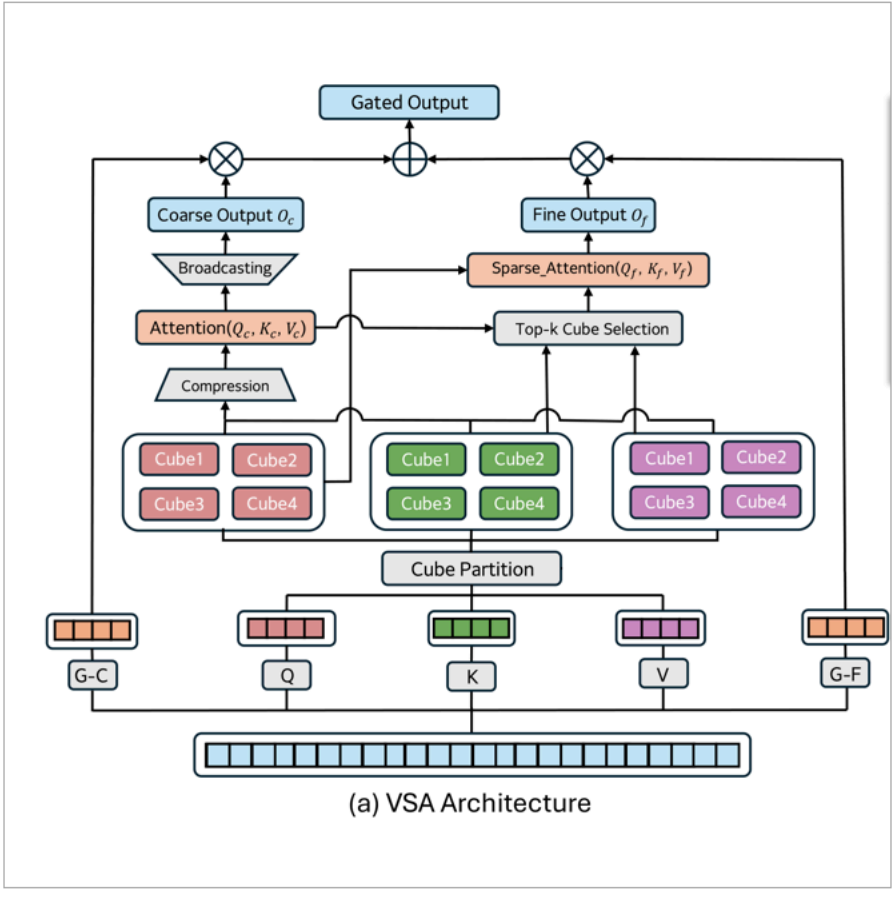
- Per-query Top-K cube scores choose which K/V regions to visit

Spend fine attention on selected detail

- Compute token attention inside the chosen blocks; skip the rest

Keep global context and local detail

- Learned gates combine coarse and fine outputs



(a) VSA Architecture

$$O = O_c \odot G_c + O_f \odot G_f$$

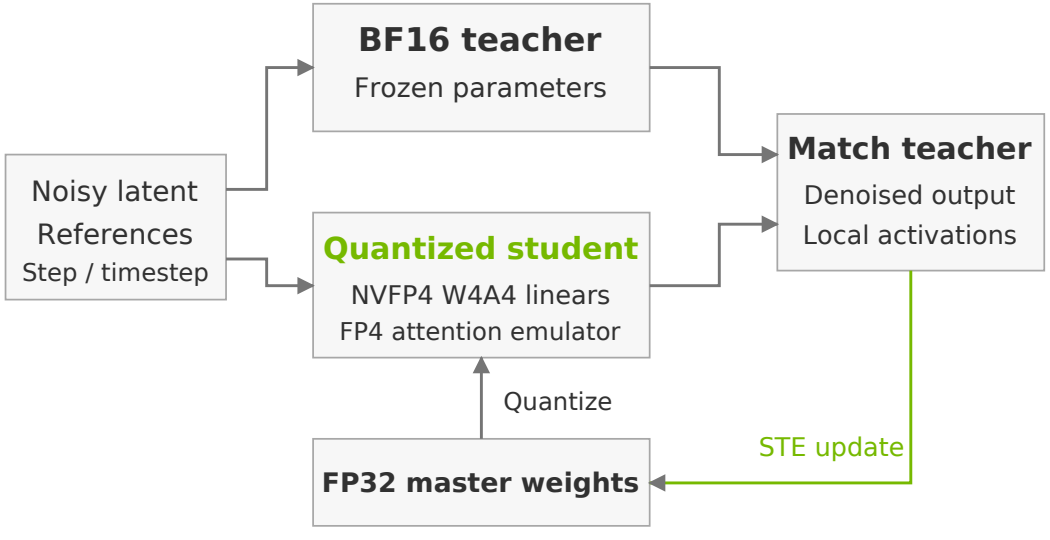
Global coarse context + selected fine detail · FastH3 adaptation: G_f = 1

Source: Video Sparse Attention, arXiv:2505.13389 · Figure 1 and Section 3 · H3 uses logical tiles of 128

FastH3 Omni: Quantization-Aware Distillation

An eight-forward student learns to compensate for quantization error before deployment

Learn with quantization in the forward pass



What runs in four bits

Module	Forward	Update
Q / K / V / O	NVFP4	Train
FFN projections	NVFP4	Train
VSA gates	NVFP4	Frozen
Norms	Original	Frozen
Boundary modules*	FP32	Frozen
Fine attention	FP4 emulator	Backprop

*Designated input / output / time modules

Execution precision ≠ trainable parameters

Few-step + four-bit inference

Eight model evaluations reduce sampling work. NVFP4 linears reduce numerical precision.

Train through quantization

FP32 master weights update through STE gradients while the forward executes NVFP4.

Deploy the student only

Teacher supervision and reconstruction losses recover quality during training; inference omits them.

Source: FastVideo PR #1989 · public OmniRef recipe @ ea3275d · implementation analysis, not a reproduced training result

Quantization: Three-Way Video Comparison

BF16 reference · Direct Online NVFP4 · Published QAD checkpoint

BF16 Reference



Original BF16 weights

Request E2E: 48.12 s*

LPIPS ↓ 0.0000 | SSIM ↑ 1.0000

Online NVFP4



No calibration / fine-tuning / QAD

Request E2E: 32.44 s*

LPIPS ↓ 0.2978 | SSIM ↑ 0.5778

QAD NVFP4



Published QAD checkpoint

Request E2E: 28.40 s*

LPIPS ↓ 0.2502 | SSIM ↑ 0.6438

1344 × 768 · 124 frames @ 24 FPS · 8 steps · Seed 1234

*Single requests; warmup / encoder / HSDP differ. Metrics: all frames vs. BF16.

Matched request · cuDNN BF16 attention + original VAE · checkpoint comparison, including encoder precision

QAD: Recover Video and Audio

Match denoised outputs and local activations while keeping VSA gate weights frozen

1 Denoised output

Match student x0 to teacher x0

Normalize each sampling step and modality by its initial PTQ error

Video and audio carry equal weight

2 Layer reconstruction

Give each student unit the teacher's input

Correct attention and FFN error locally, with per-block backward

Video : audio : conditioning
1 : 1 : 0.25

3 Student rollout

Train on the student's own trajectory

After step 20, use student rollout on 25% of rows

Reduce exposure to teacher-only states

Day-0 deployment: preserve packed weights, independent projection scales and TP scale layout

Source: pdd_qad.py + pdd_qad_recon.py + OmniRef YAML @ ea3275d · public recipe, not our training benchmark

Ulysses

Each GPU swaps a token shard for seven full-sequence heads

Start with token shards

- Eight GPUs split 56,960 tokens; each rank keeps 7,120 tokens and all 56 heads

Swap the sharding axis

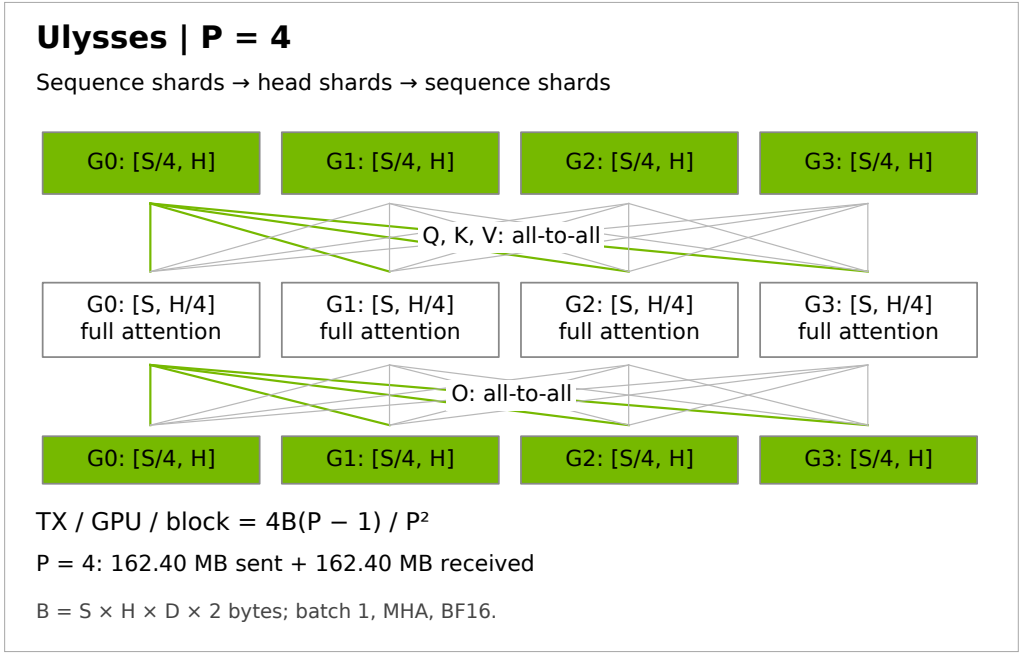
- Q/K/V/G all-to-all changes [S/8, 56] into [S, 7]

Do the expensive part locally

- Each GPU sees the full sequence for its own seven heads

Switch back for the block

- The O all-to-all restores token shards for projection, residual, and MLP



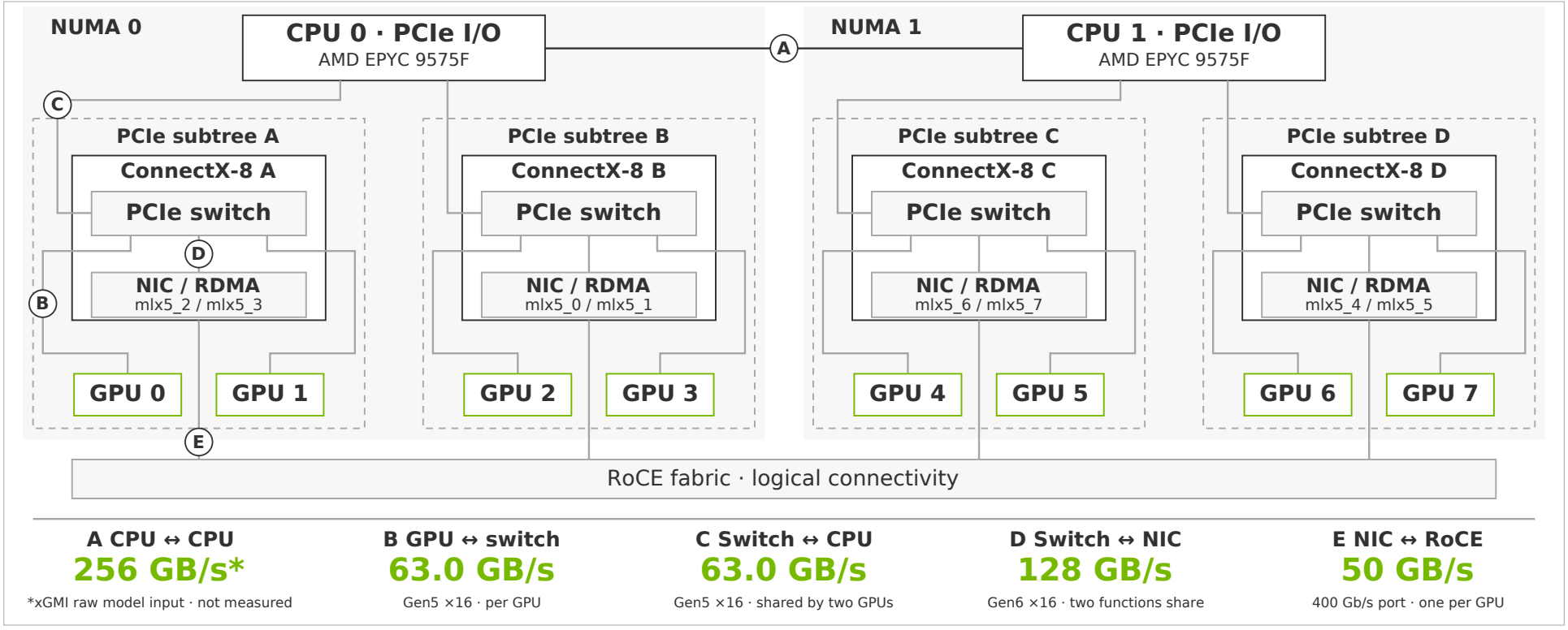
Sequence shards → head shards → sequence shards

Original website illustration: P = 4
Measured H3 configuration at left: P = 8

Source: retained scaling website · Ulysses illustration P=4 · H3 profile at left P=8

Eight-GPU Topology

Two CPU sockets · four ConnectX-8 cards · two GPUs per PCIe subtree



One-way link capacities · network ports support simultaneous TX and RX

Sources: local PCIe / port audit · NVIDIA ConnectX-8 · AMD EPYC 9005 architecture · xGMI raw capacity assumption

Local NCCL to RDMA

Matched eight-GPU Q/K/V/O exchanges · FI RDMA is 2.33–2.47× faster across three sequence lengths

Local NCCL: one P2P peer, six SHM peers

- Cross-subtree payload uses host memory
- Two GPUs share one 63.0 GB/s uplink
- $2 \times B \times 6/7 \leq 63.0 \rightarrow B \leq 36.8$ GB/s

Cross-NUMA: C = 256 GB/s (raw model)

- $4 \times B \times 4/7 \leq 256 \rightarrow B \leq 112$ GB/s
- Local ceiling: min(36.8, 112) = 36.8

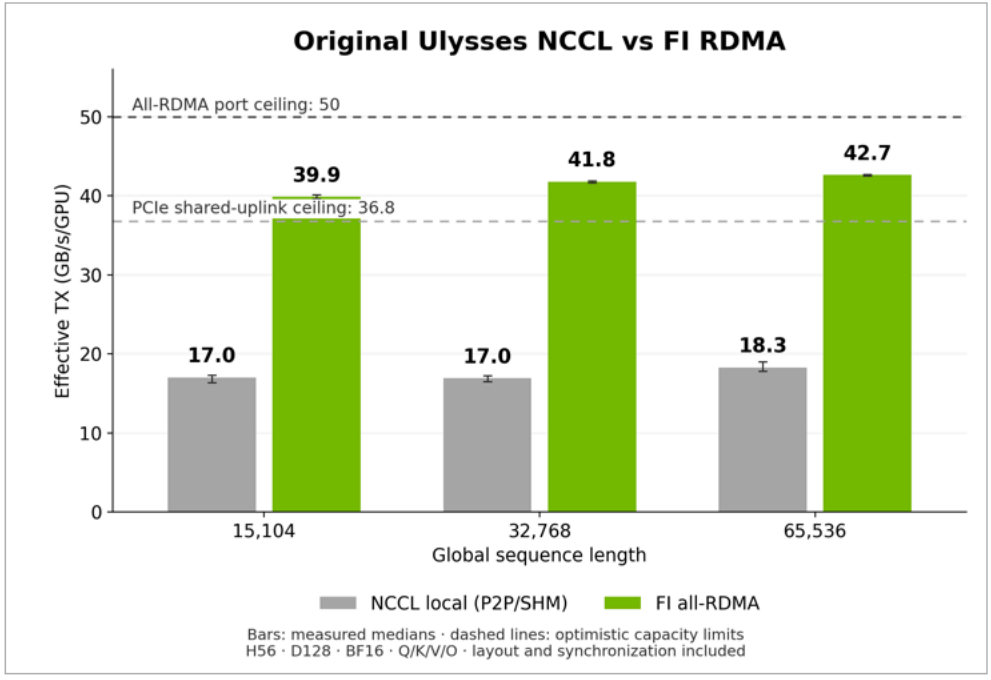
FI RDMA: one nearest 400G port per GPU

- Bypasses host buffers and CPU-facing uplinks
- $B \leq \min(63.0, 128/2, 50) = 50$ GB/s

Shared PCIe uplink is the tighter raw bound

SHM, layout and scheduling add further cost.
Next: hide waits with 2 / 3 / 2 head bands,
then reduce compute launch cost with graphs.

B: off-rank TX per GPU · C: raw xGMI model input
Necessary link bounds; effective capacity can be lower



Measured: communication + layout + scheduling; not E2E.

Separate Q/K-first candidate: 96/96 rank checks exact
on four GPUs; full eight-GPU H3 validation pending.

Original vLLM-Omni Ulysses

Q/K/V: pack head shards → NCCL all-to-all → restore sequence layout

```
# comm.py:44–62 · original Q/K/V branch
input_t = (
    input.reshape(bs, shard_seqlen,
                  seq_world_size, shard_hc, hs)
    .transpose(0, 2).contiguous()
)
output = torch.empty_like(input_t)
dist.all_to_all_single(output, input_t, group=group)
output = output.reshape(seqlen, bs, shard_hc, hs)
output = (output.transpose(0, 1).contiguous()
          .reshape(bs, seqlen, shard_hc, hs))
```

Four exchanges per dense block

- Q / K / V: scatter heads
- O: scatter sequence

NCCL selects the route

- P2P / SHM / NET depend on topology and configuration

At batch = 1

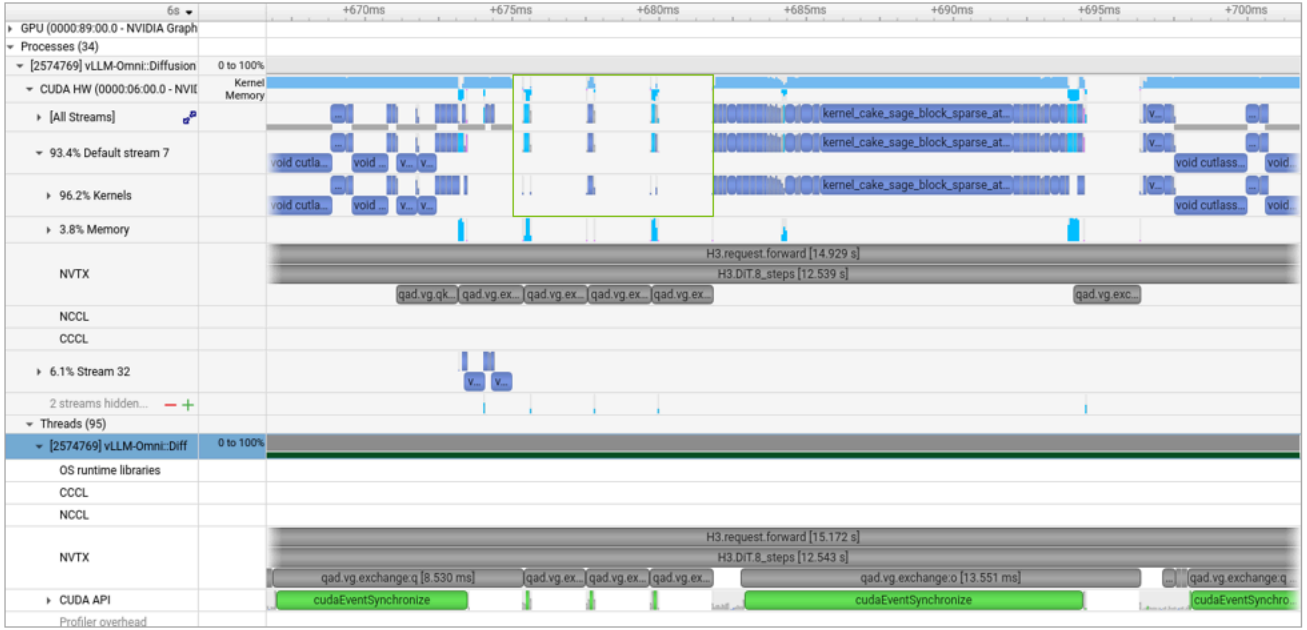
- Q/K/V output transpose is a view; contiguous() does not always copy

ulysses.py:538–547 → SeqAllToAll4D.forward → all_to_all_4D → PyTorch NCCL
O reverses the exchange at ulysses.py:721 · pinned source: 63487a7f22db

Source: Base50 checkout 63487a7f22db · comm.py:44–62 · line wrapping adjusted; guards omitted · Apache-2.0

RDMA Still Leaves Ulysses Waiting

Faster transfers leave a dependency bubble when all heads move as one group



Why bubbles remain

Attention waits for complete Q/K/V.
Output projection waits for the returned O.

3.24 s / GPU

Exposed transport phase
8-GPU mean, full DiT trace

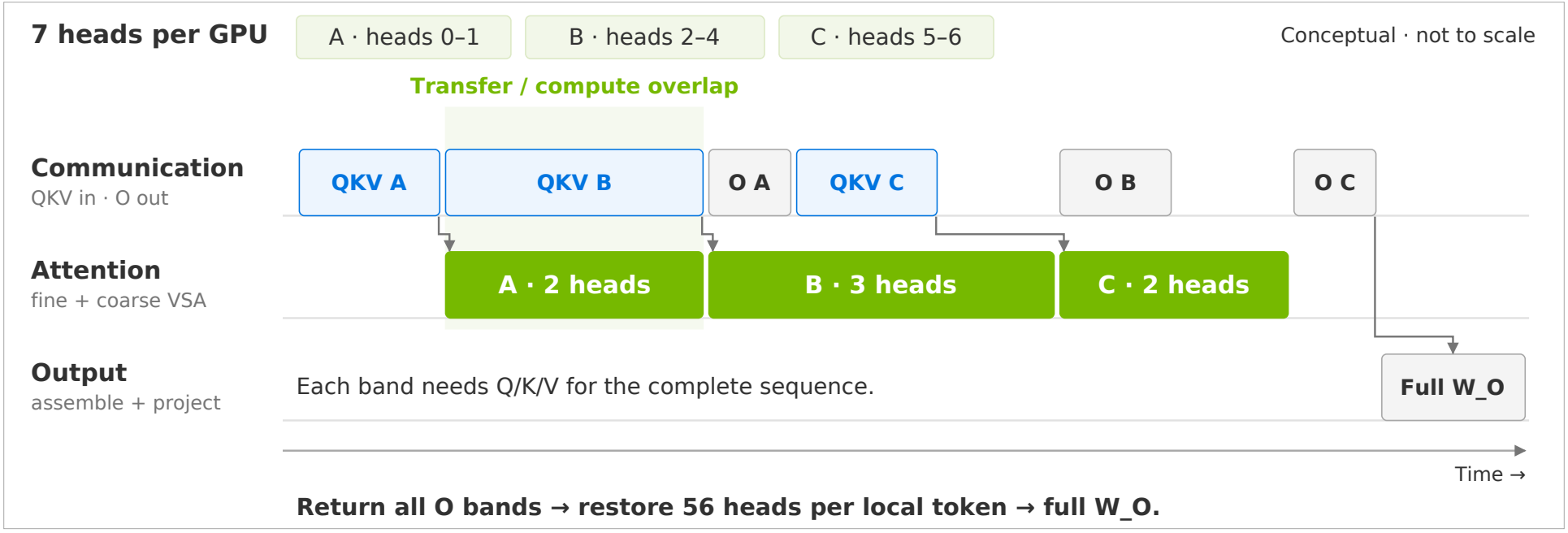
Compute-lane gaps overlap communication phases; NIC work need not appear as a CUDA kernel.

Transport phase includes host posting / CQ polling / flush. This is a profiled baseline, not a 2/3/2 result.

Nsight Systems 2025.3.2 · retained FI RDMA baseline · 8 × RTX PRO 5000 · 960 × 544 · 362 frames · 8 steps

Pipeline Ulysses by Head Bands

Use 2 → 3 → 2 heads per GPU to overlap communication and attention



Start earlier

Run the first two heads as soon as their full-sequence inputs arrive.

Overlap across bands

Transfer the next band while current-band attention executes.

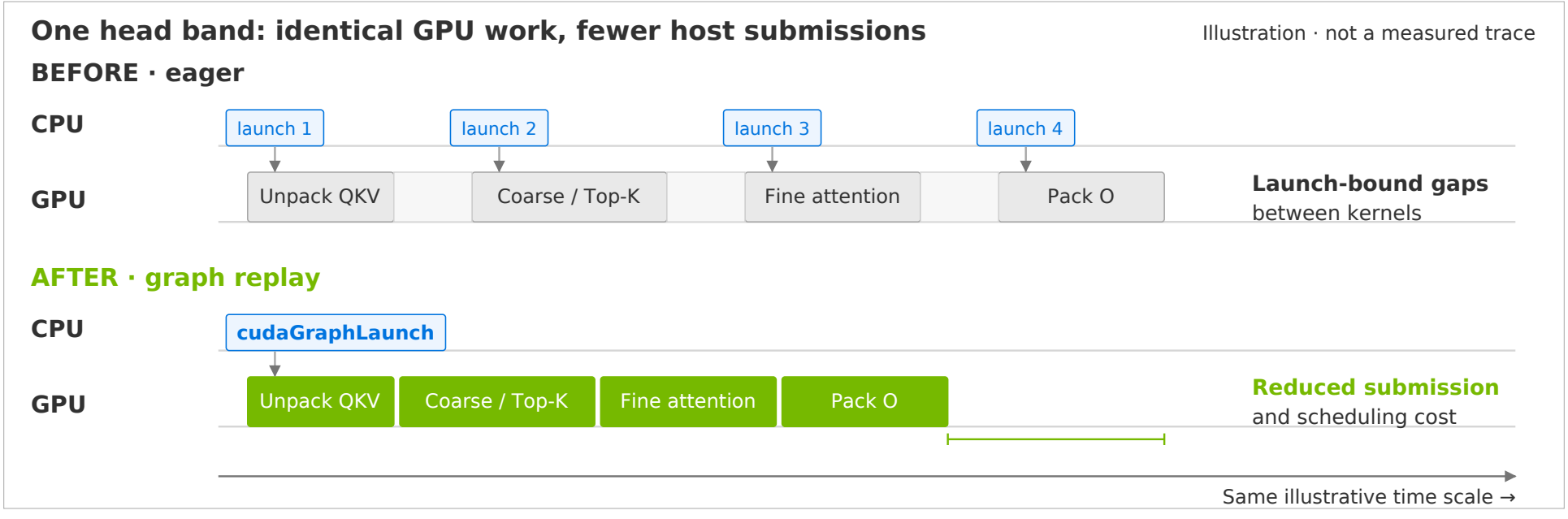
Then reduce launch cost

Repeated small compute islands motivate CUDA Graph replay.

Selected schedule: 2 / 3 / 2 · dependency schematic, not a measured Nsight timeline

CUDA Graph: Reduce Submission Gaps

After head pipelining, replay each stable compute band with one graph launch



Historical 1 / 2 / 3 / 1 · measured separately

8-GPU request: 13.250 → 11.994 s (−9.48%)

Warmed request cohorts; full output MP4 included.

Compute only: 12.323 → 12.040 ms (−2.30%)

Single GPU · unpack + VSA + pack · no RDMA

Sources: H3 compute-graph prototype and retained measurements · NVIDIA CUDA Programming Guide · conceptual timeline

Why CUDA Graph Fits H3

Fixed video geometry and stable buffers make repeated head-band compute reusable

Repeated work

50 blocks × 8 steps = 400 calls per band
for one unchanged execution profile.

Stable shapes and buffers

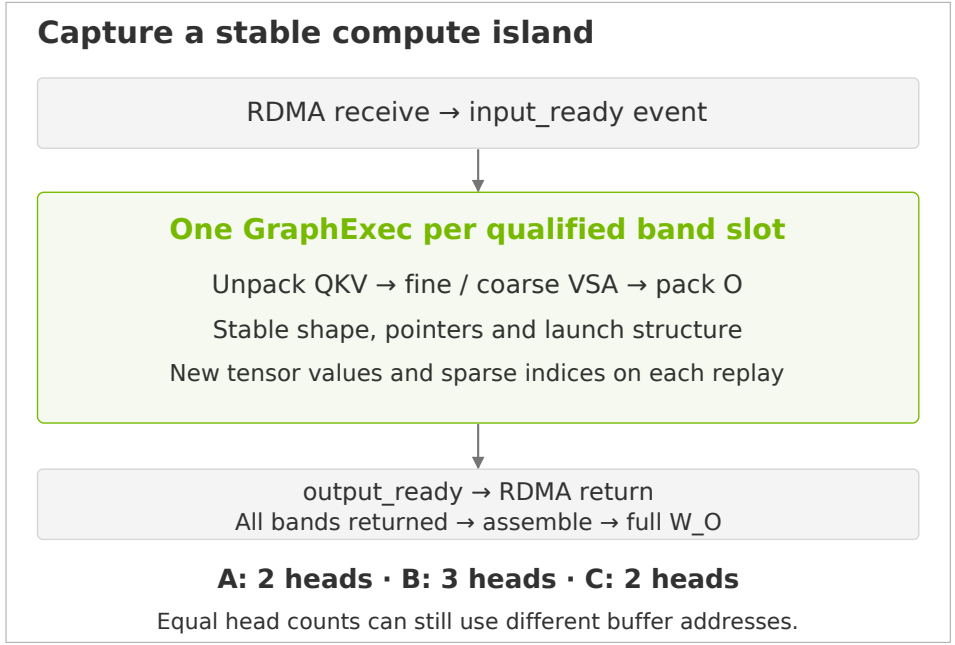
Refresh Q/K/V values in registered storage.
Keep geometry, addresses and launch structure fixed.

More launches after head splitting

Capture unpack → VSA → pack together.
Amortize capture over repeated band execution.

Reuse has conditions

New geometry needs a separately qualified graph.
Budget capture time and private-pool memory.
Compute-bound kernels may gain little.



H3: stable video / head profiles · **LLM decode: often batch / token profiles**

Source: H3 prototype capture/key audit · 50 DiT blocks × 8 evaluations · proposed production profile dispatch

Real-Time Demonstrated; Attribution Pending

Only Base50 uses the locked 960×544, 124-frame request; each optimized result uses a different workload

Status	Workload	System	Result / Limit
Measured baseline	960×544 · 124f · 5.167 s	Base50 · BF16 · USP8 · NCCL	247.956 s · RTF 47.992
15 s study	960×544 · 362f · 15.083 s	PDD8 · QAD · VSA · RDMA	14.974 s mean · RTF 0.993
10 s study	960×544 · 243f · 10.125 s	4-step Turbo · MXFP8 · Sage · RDMA	6.958 s · RTF 0.687
Portrait study	576×1024 · 119 new frames · 4.958 s	4-step MXFP8 · Sage · FP4 VAE · RDMA	4.459 s median · RTF 0.899
Required matched test	Lock image, audio, prompt, seed, shape, and request mode	PDD8 → VSA → QAD/Sage → RDMA	At least 3 runs per stage

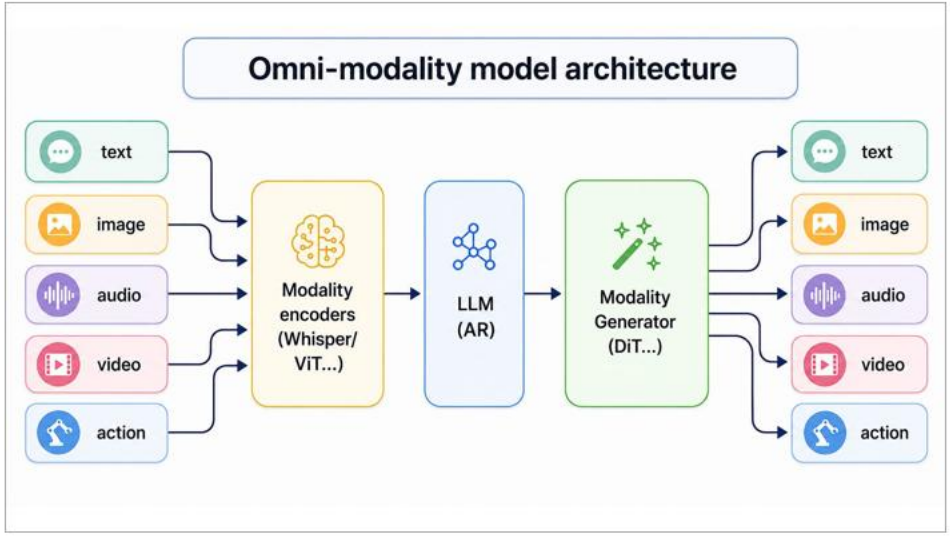
Sources: [A] Base50 r4 · [B] QAD 15 s cohort · [C] paired 10 s study · [D] portrait production · compare only within a row; timing and memory methods differ

02 · ComfyUI Apps With vLLM-Omni



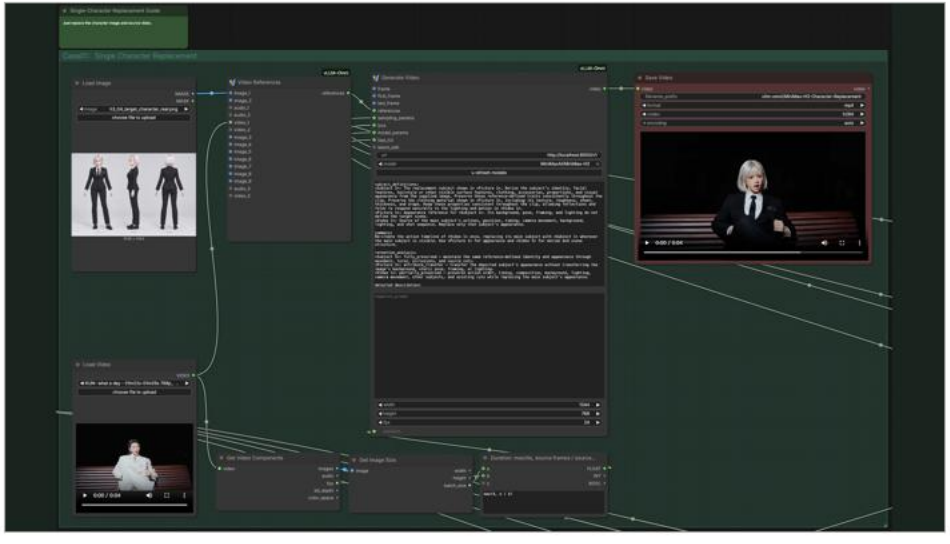
vLLM-Omni Brings Multi-GPU H3 to ComfyUI

Open PR #9194 adds character replacement, continuation-aware references, and chainable LoRA controls



Official vLLM-Omni Architecture

ComfyUI authors the workflow; vLLM-Omni runs scheduling, model execution, and multi-GPU deployment



ComfyUI Workflow · PR #9194

The workflow supports single- and multi-character replacement, source-derived duration, long-video continuation, and LoRA controls

Sources: vLLM-Omni v0.26 public deck, slide 22 · vLLM-Omni PR #9194 (open)

